

Pawnbroking

New Shop Onboarding Runbook

From the first real shop install is built in — cloud provisioning, sync agent, one-time backfill, PK-collapse fixes, image

PAWNBROKING — NEW SHOP ONBOARDING RUNBOOK

Battle-tested step-by-step (every gotcha from the alwarpuram install is baked in here so the next shop goes smoothly).

Replace these placeholders throughout:

<SHOPID> lowercase shop key, e.g. balamurugan

<EMAIL> the owner's gmail for mobile login

<PGPASS> the PostgreSQL password set during install on the shop PC

Three setup targets:

CLOUD (Railway) — you, remotely, once per shop

SHOP PC (Windows) — at the shop: PostgreSQL + Desktop app + Sync agent

PHONE (Android) — install the one universal APK, sign in by email

IMPORTANT context that bit us on alwarpuram — read once:

- The Magizhchi cloud SQL console SPLITS on ';'. It CANNOT run DO \$\$... \$\$ blocks or multi-statement functions. Run those in pgAdmin on the shop PC, OR as single statements on the console.
- The sync agent reads its config ONLY from:

C:\ProgramData\PawnBroking\sync.properties

NOT from the install folder. ProgramData is hidden.

- Existing rows do NOT auto-sync. Triggers only capture NEW changes.

A one-time backfill is required for historical data.

- Some desktop tables collapse on the cloud unless their real primary key is used. company_billing + repledge_billing both needed attention. The fix is built into the steps below.
- Images and the database are SEPARATE. Restoring the DB does not bring the image files — copy the image + backup folders too.

PART 1 — CLOUD PROVISIONING (Railway → Data → Query)

Run these as SINGLE statements (console splits on ';').

1.1 Register the tenant:

```
INSERT INTO public.tenants (shop_id) VALUES ('<SHOPID>') ON CONFLICT DO NOTHING;
```

1.2 Allow the owner's email to log in:

```
INSERT INTO public.tenant_primary_email (shop_id, email)
VALUES ('<SHOPID>', '<EMAIL>') ON CONFLICT DO NOTHING;
```

1.3 Grant multi-shop access (lets this email see the shop in the app):

```
INSERT INTO public.user_shop_access (email, shop_id, role)
VALUES ('<EMAIL>', '<SHOPID>', 'OWNER') ON CONFLICT (email, shop_id) DO NOTHING;
```

1.4 Generate the sync API key (COPY the result — you can't retrieve it later):

```
INSERT INTO public.shop_credentials (api_key, shop_id, label)
VALUES ('mbk_' || replace(gen_random_uuid()::text, '-', ''))
      || replace(gen_random_uuid()::text, '-', ''),
      '<SHOPID>', '<SHOPID> shop - sync agent')

RETURNING api_key;
```

1.5 Add the shop to Railway's TENANTS variable:

Railway → service → Variables → TENANTS → append ,<SHOPID>

Save → it auto-redeploys (~3 min). Confirm in Deploy Logs:

```
"Provisioning tenant schema '<SHOPID>'"
```

PART 2 — SHOP PC: PREREQUISITES

2.1 Install Java 17:

<https://adoptium.net> → Temurin 17 → Windows x64 .msi

2.2 Install PostgreSQL 16:

<https://www.postgresql.org/download/windows/>

- Port 5432 (default)
- SET A PASSWORD and WRITE IT DOWN → this is <PGPASS>

2.3 Verify (Command Prompt):

```
java -version          → shows 17.x
(pgAdmin opens & connects with <PGPASS>)
```

PART 3 — SHOP PC: DESKTOP APP + DATA RESTORE

3.1 Run 2-DesktopApp\PawnBrokingSetup.exe → enter <PGPASS> when asked.

If migrating an existing shop, restore their PostgreSQL backup now

so company_billing etc. are populated.

3.2 COPY THE IMAGE + BACKUP FOLDERS from the old machine (the DB

restore does NOT include these files):

- bill/customer images → whatever path is in
company_other_settings.camera_temp_file_name (e.g. D:\IMAGES)
- backup files → whatever path is in
company.backup_file_path (e.g. D:\<shop> backup)

Put them at the SAME paths the DB columns point to (or update the
columns to the new paths).

3.3 Confirm the paths the agent will read (pgAdmin on shop PC):

```
SELECT company_id, camera_temp_file_name FROM company_other_settings;
SELECT id, backup_file_path FROM company;
```

Open both in File Explorer — they must exist and contain files.

PART 4 — SHOP PC: SYNC AGENT

4.1 Copy the 3-SyncAgent folder to:

```
C:\Program Files\PawnbrokingSync\
```

4.2 Create the config at the FIXED path the agent reads

(NOT in the install folder). Admin Command Prompt:

```
mkdir "C:\ProgramData\PawnBroking"
notepad "C:\ProgramData\PawnBroking\sync.properties"
```

Paste, filling in <PGPASS>, <SHOPID>, and the API key from 1.4:

```
db.url=jdbc:postgresql://localhost:5432/pawnbroking
db.user=postgres
db.password=<PGPASS>

cloud.url=https://devpawn.magizhchi.academy
cloud.api_key=mbk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
shop.id=<SHOPID>

batch.size=200
poll.interval.ms=5000
health.port=17654
schema.guard.interval.ms=30000
image.scan.interval.ms=60000
backup.retention.days=30
```

Save, close.

4.3 Install + start the service (admin Command Prompt):

```
cd "C:\Program Files\PawnbrokingSync"
pawnbroking-sync.exe install
pawnbroking-sync.exe start
```

4.4 Confirm healthy (err.log empty, out.log shows activity):

```
type pawnbroking-sync.err.log
type pawnbroking-sync.out.log
```

Common errors:

```
"Cannot read config"           → file not at the ProgramData path
"password authentication failed" → wrong <PGPASS>
"tenant not configured"/401     → Part 1 not done / wrong api_key
```

PART 5 — ONE-TIME BACKFILL OF EXISTING DATA

Run in pgAdmin on the SHOP PC (handles DO blocks; console can't).

Skip entirely if this is a brand-new shop with no history.

The triggers only capture FUTURE changes. To push existing rows, re-emit them through the (PK-aware) trigger.

5.1 FIRST fix tables that lack a usable primary key. repledge_billing

historically has none → it would collapse to 1 row on the cloud.

Check + add (pgAdmin):

```
-- is repledge_bill_id unique & non-null?
SELECT count(*) total, count(DISTINCT repledge_bill_id) d,
       count(repledge_bill_id) nn FROM repledge_billing;
-- if total = d = nn, add the PK:
ALTER TABLE repledge_billing ADD PRIMARY KEY (repledge_bill_id);
-- (if it errors "multiple primary keys", it already has one – skip)
```

5.2 Make sure the PK-aware trigger function is installed. Paste this

WHOLE block in pgAdmin (it is idempotent):

```
CREATE OR REPLACE FUNCTION sync_capture() RETURNS trigger AS $$
DECLARE
    v_shop_id TEXT; v_payload JSONB; v_row_pk TEXT;
    v_pk_cols TEXT[]; v_col TEXT; v_parts TEXT[] := ARRAY[]::TEXT[];
BEGIN
    BEGIN v_shop_id := current_setting('app.shop_id', true);
    EXCEPTION WHEN OTHERS THEN v_shop_id := NULL; END;
    IF v_shop_id IS NULL OR length(v_shop_id)=0 THEN v_shop_id:='DEFAULT'; END IF;
    IF TG_OP='DELETE' THEN v_payload:=to_jsonb(OLD); ELSE v_payload:=to_jsonb(NEW); END IF;
    SELECT array_agg(a.attname ORDER BY array_position(i.indkey::int[]::int[], a.attnum))
        INTO v_pk_cols FROM pg_index i
        JOIN pg_attribute a ON a.attrelid=i.indrelid AND a.attnum=ANY(i.indkey)
        WHERE i.indrelid=(TG_TABLE_SCHEMA||'.'||TG_TABLE_NAME)::regclass AND i.indisprimary;
    IF v_pk_cols IS NOT NULL AND array_length(v_pk_cols,1)>0 THEN
        FOREACH v_col IN ARRAY v_pk_cols LOOP
            v_parts := array_append(v_parts, COALESCE(v_payload->>v_col,''));
        END LOOP;
        v_row_pk := array_to_string(v_parts,'|');
    ELSE
        v_row_pk := COALESCE(v_payload->>'id', v_payload->>'bill_no',
            v_payload->>'bill_number', v_payload->>'customer_id',
            v_payload->>'company_id', v_payload->>'pk');
    END IF;
    INSERT INTO sync_outbox(event_id,shop_id,table_name,op,row_pk,payload)
    VALUES (gen_random_uuid(), v_shop_id, TG_TABLE_NAME, LEFT(TG_OP,1), v_row_pk, v_payload);
    PERFORM pg_notify('sync_channel', gen_random_uuid()::text);
    RETURN COALESCE(NEW, OLD);
END; $$ LANGUAGE plpgsql;
```

5.3 Make sure triggers are attached to every business table (idempotent):

```
DO $$
DECLARE t TEXT;
BEGIN
    FOR t IN SELECT table_name FROM information_schema.tables
        WHERE table_schema=current_schema() AND table_type='BASE TABLE'
        AND table_name NOT IN ('sync_outbox','sync_outbox_dlg')
        AND table_name NOT LIKE 'flyway_%' AND table_name NOT LIKE 'pg_%'
    LOOP
        EXECUTE format('DROP TRIGGER IF EXISTS trg_sync_%I ON %I', t, t);
        EXECUTE format('CREATE TRIGGER trg_sync_%I AFTER INSERT OR UPDATE OR DELETE '
            ||'ON %I FOR EACH ROW EXECUTE FUNCTION sync_capture()', t, t);
    END LOOP;
END $$;
```

5.4 Re-emit existing rows. Set the shop id, then no-op UPDATE each table

(pgAdmin — run the SET and UPDATES together in one session):

```
SET app.shop_id = '<SHOPID>';
TRUNCATE sync_outbox;

UPDATE company SET company_id = company_id;
UPDATE company_billing SET company_id = company_id;
UPDATE customer_details SET company_id = company_id;
UPDATE repledge_billing SET company_id = company_id;
UPDATE company_advance_amount SET company_id = company_id;
UPDATE company_todays_account SET company_id = company_id;
UPDATE company_todays_account_available_amount SET company_id = company_id;
UPDATE company_other_debit SET company_id = company_id;
UPDATE company_other_credit SET company_id = company_id;

NOTIFY sync_channel, 'backfill';
```

(Add any other tables the shop uses the same way.)

5.5 Watch it drain (pgAdmin). This is the REAL backlog:

```
SELECT count(*) FROM sync_outbox WHERE sent_at IS NULL;
```

It counts DOWN to 0 as the agent ships batches. Leave the PC on.

NOTE: count(*) of the WHOLE table grows (sent rows are kept) —

always filter sent_at IS NULL for the true pending count.

PART 6 — VERIFY DATA ON THE CLOUD (Railway)

6.1 Total bills landed (should equal desktop's company_billing count):

```
SELECT count(*) FROM <SHOPID>.projections
WHERE table_name='company_billing' AND NOT deleted;
```

6.2 Status breakdown — compare against the desktop. If a table shows

far FEWER distinct rows than the desktop, it collapsed (recheck its

PK in 5.1):

```
SELECT upper(payload->>'status') s, count(*)
FROM <SHOPID>.projections
WHERE table_name='company_billing' AND NOT deleted
GROUP BY 1 ORDER BY 2 DESC;
```

IN-STOCK bills the app shows = OPENED + LOCKED. If these match the

desktop, the Stock screen count is correct.

COLLAPSE CHECK for any table that looks short:

```
-- desktop (pgAdmin): is the composite key unique?
SELECT count(*) total,
       count(DISTINCT (company_id, jewel_material_type, bill_number)) d
FROM company_billing;
-- if total=d but cloud has fewer, the trigger was old when you
-- backfilled → re-run 5.2 then 5.4 for that table.
```

PART 7 — IMAGES & BACKUPS

Image sync needs the cloud to hold the shop's box token. That token is

auto-minted the FIRST time someone OTP-logs-in on the phone for this

shop. So do Part 8 (phone login) at least once, THEN images flow.

7.1 Folder layout the agent understands (under camera_temp_file_name root):

```
BILLS:      <root>\<CMP>\<MATERIAL>\<billNo>\<image>.png
CUSTOMERS:  <root>\...\CUSTOMERS\<customerId>\<image>.png
```

(A folder named CUSTOMERS or CUSTOMER anywhere under the root is

treated as customer photos.)

7.2 Watch progress (shop PC, pgAdmin):

```
SELECT material, count(*) FROM sync_image_uploads GROUP BY material;
SELECT count(*) FROM sync_backup_uploads;
```

Full first-time image sync of a large shop takes a few HOURS

(throttled to ~54/min for the box). Leave the PC running.

7.3 Backups: only files modified within backup.retention.days (default

30) are uploaded, so old daily dumps don't flood the box. Large

backups (100MB+) are streamed — no size error.

7.4 Verify on cloud (Railway):

```
SELECT count(*) FROM <SHOPID>.bill_images;  
SELECT count(*) FROM <SHOPID>.backup_files;
```

PART 8 — PHONE (the one universal APK)

8.1 Install 4-MobileApp\Pawnbroking.apk on the phone (allow "unknown sources"). Same APK for every shop — no rebuild.

8.2 Open Pawnbroking → enter <EMAIL> → Send OTP → type the 6-digit code from the email → Verify.

- If the email has ONE shop → lands on Home for that shop.
- If the email has MULTIPLE shops → a picker appears; tap one.
Switch anytime via Home → 3-dot menu → Switch Shop.

8.3 The Home toolbar subtitle shows the active shop. Check:

- Stock Details → ALL DETAILS shows in-stock (OPENED+LOCKED) count
- Today's Account → defaults to last L-marker date + 1
- MIS Report → last 12 months
- Bill detail → bill images (once uploaded)

8.4 "Too Many Requests" on Send OTP = box rate limit. Wait 60s.

QUICK TROUBLESHOOTING

Agent won't start / "Cannot read config"

→ sync.properties must be at C:\ProgramData\PawnBroking\ (hidden)

Agent "password authentication failed"

→ wrong db.password; use the value pgAdmin connects with

Cloud rejects events / "tenant not configured" / 401

→ Part 1 incomplete, or wrong api_key, or shop not in TENANTS var

App shows far fewer bills than desktop

→ a table collapsed; recheck its PK (Part 5.1) and re-backfill (5.4)

App shows ■0 on Today's Account

→ company_todays_account / other_debit / other_credit not backfilled

→ re-run their UPDATES in 5.4

Images not syncing

→ no phone login yet (token not minted) → do Part 8 once

→ wrong image root → check camera_temp_file_name path exists on PC

→ old agent jar → copy the latest from 3-SyncAgent and restart service

Backups fail with 413

→ old cloud build; ensure Railway is on the latest deploy

sync_outbox count keeps GROWING

→ you're counting all rows; filter "WHERE sent_at IS NULL" for the

true pending backlog (sent rows are retained, not deleted)

HANDOFF CHECKLIST — done when:

- ☐ Tenant + email + access + api_key created on cloud (Part 1)
- ☐ Shop added to Railway TENANTS var, schema provisioned
- ☐ Java 17 + PostgreSQL 16 installed (Part 2)
- ☐ Desktop app running; DB + image + backup folders in place (Part 3)
- ☐ Sync agent service running, err.log clean (Part 4)
- ☐ Backfill drained: sync_outbox WHERE sent_at IS NULL = 0 (Part 5)
- ☐ Cloud company_billing count == desktop count (Part 6)
- ☐ Phone signs in, shows correct in-stock count (Part 8)
- ☐ Images + backups uploading (Part 7) — may finish over a few hours
- ☐ Owner given: <EMAIL>, <PGPASS>, support contact